

DMT: Assignment 2

Group 14: Sebastiaan Broos^[2645847], Job van den Akker^[2650043], and Nika
Kovačić^[2862120]

Vrije Universiteit, Amsterdam, NL

1 Introduction & Business Understanding

Predicting which hotel a user is most likely to click on is important for different actors in the travel ecosystem. For end users, personalized ranking of search results means they spend less time sifting through irrelevant options and more quickly find accommodations that suit their needs, leading to higher satisfaction. For booking platforms such as Expedia and Booking.com, improving click-through and booking rates directly drives revenue growth, reduces customer acquisition costs, and enhances competitive differentiation.

In this assignment, we built different types of models to try to get the best accuracy score from a list of hotel properties, evaluated with Normalized Discounted Cumulative Gain @5 (NDCG@5)[1]. We are doing this in a competition, hosted on Kaggle, where we also got the dataset from, competing against other VU student-teams[2].

To do this, we first researched how similar problems have been approached in the past. We reviewed the kernels and discussion threads from the Expedia Hotel Recommendations Kaggle competition [3]. Many top-scoring teams used gradient-boosted tree frameworks, particularly LightGBM and XGBoost, as well as some CatBoost methods [4][5]. Beyond Kaggle, we also looked at industry applications; for instance, Expedia has developed lodging ranking models to personalize and optimize hotel search results for users [6]. These models aim to rank retrieved hotels based on predicted user preferences. Guided by this research and the content from the Data Mining course, we implemented two distinct types of models. The first one is an ensemble combining different gradient-boosting frameworks. For the second model we chose CatBoost with Collaborative Filtering (CF).

2 Data Understanding

An exploratory data analysis of the Expedia hotel search dataset was conducted to understand its structure, completeness, and key characteristics. The training dataset included 4958347 rows and 54 columns, while the test set had 4959183 rows and 50 columns, excluding *booking_bool*, *click_bool*, *gross_bookings_usd* and *position* due to these variables being either target variables or leakage data. The dataset contains information regarding a specific search. One search contains information about users and properties that are shown, such as what ranking a property has or number of rooms to book. There is a lot of unique searches and hotel properties, with users and hotels spanning over 200 and 170 countries respectively. Table 1 shows some basic data properties and summaries of the present features in the training set.

Property	Value / Notes
Num. of unique search IDs	199,795 (138,390 booked)
Num. of unique property IDs	129,113
Num. of countries users are from	210
Num. of countries properties are located	172
Data types	33 numerical, 21 categorical/integer, 1 temporal (representing timestamps)
Notable issues	a lot of missing values, sparse features, more features in the train set than test (leakage variables)
Correlation between click and book	0.78
Interesting observations	– Average times a property appears is 38.4 ($std = 81.89$, $median = 12$) – properties appearing only once (12,868) – Average num. of property results per search is 24.82 ($std = 9.11$, $median = 29$)

Table 1: Exploratory Data Analysis Summary (Train set)

2.1 Click and Booking Behavior

To understand the class distribution and behavior of the target variables, the frequency and co-occurrence of user clicks and bookings were analyzed. In the training data, 4.47% of properties were clicked, and only 2.79% resulted in a booking, while majority of properties were neither clicked nor booked (95.53%). These results revealed the class imbalance shown in Figure 1. Additionally, strong positive correlation ($r = 0.78$) was found between `click_bool` and `booking_bool`, indicating that clicks are a useful signal when predicting bookings.

Bar plots in Figure 2 show how click and booking rates vary with hotel star ratings and review scores. Both trends increase with higher star ratings and better reviews, peaking around 4 stars and a review score of 4.0–4.5, before slightly declining, indicating that hotel quality could have a significant impact on user behavior.

2.2 Missing Values

Multiple features in the training set showed a solid number of missing values (Figure 3). Primarily, `visitor_hist_adr_usd` and `visitor_hist_starrating` are missing in over 90% of the entries, possibly because many users lack prior booking history. Similarly, the dataset contains many features related to competitors' data, such as `comp6_rate` and `comp1_inv`. An analysis of these features showed that they were quite sparse, with most having over 80% missing values.

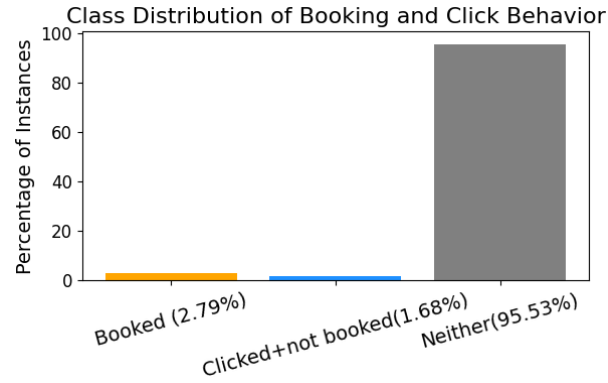


Fig. 1: Class Distribution of Booking and Click Behavior.

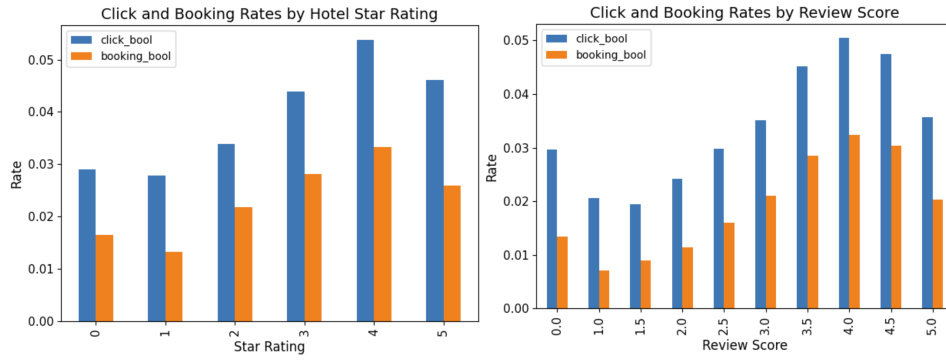


Fig. 2: Click and booking rates by hotel star rating (left) and Click and booking rates by by review score (right).

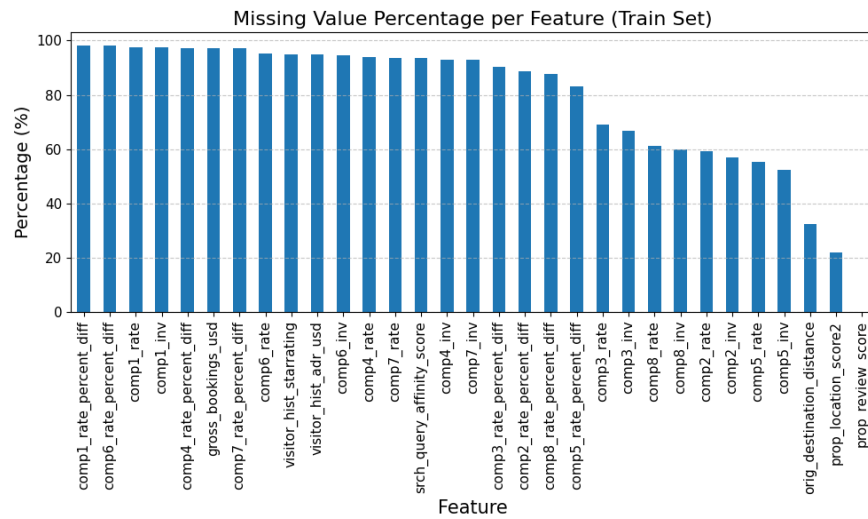


Fig. 3: Histogram of Missing Values in the Train set.

3 Data Preparation

To prepare for modeling, we created a two-branch pre-processing pipeline, as illustrated in Figure 4. Both branches start by loading the raw CSV files and applying imputation, where numeric features are filled using the median, and ID fields are assigned sentinel values. In **Branch A (Base)**, we created engineered features (e.g., search-relative price ranks and value-for-money scores), and computed popularity scores that were transferred to the test set. **Branch B (CF-Enhanced)** follows the same initial steps but extends the pipeline by generating and merging collaborative filtering (CF) scores based on historical destination-specific user behavior. Both pipelines are later evaluated and compared through a shared feature selection framework.

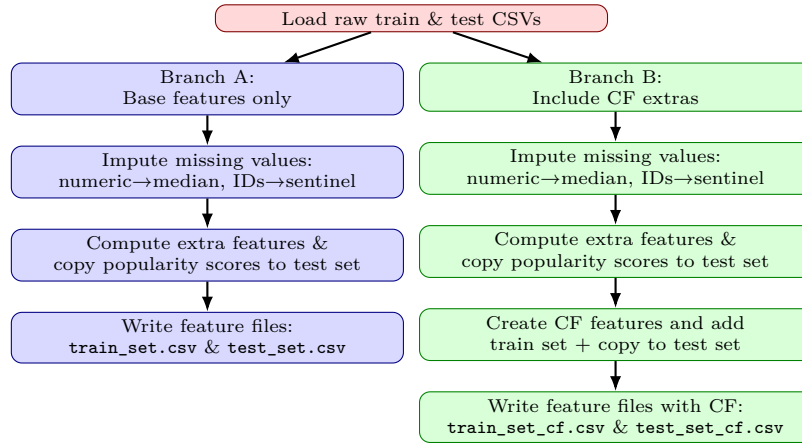


Fig. 4: Overview of two feature-engineering strategies. **Branch A** applies imputation and core feature engineering. **Branch B** follows the same steps and additionally integrates destination-specific CF scores.

3.1 Imputation

Missing values were handled using a combination of median imputation for continuous numeric features such as `price_usd` and `orig_destination_distance`, and categorical ID fields (e.g., `visitor_location_country_id`) were filled with a sentinel value of `-1`. Median imputation was chosen for its robustness to outliers, while sentinel values allow the model to recognize and learn from missingness patterns. The process was the same for both train and test sets.

3.2 Feature Engineering

Additional features were created to capture search-relative competitiveness, perceived value, and popularity. They are shown in Table 2. All the engineered fea-

tures were computed for both training and test sets, excluding the leaky features (e.g., `position`, `booking_bool`) to avoid overfitting.

Feature	Description	Pipeline
<code>price_rank</code>	Rank of price within each search group (1 = cheapest)	Base & CF
<code>star_rank_pct</code>	Percentile rank of property star rating within search	Base & CF
<code>value_score</code>	Review score divided by price (value-for-money metric)	Base & CF
<code>price_rel_search_rank</code>	Rank of price relative to median within search (pct)	Base & CF
<code>star_diff_search</code>	Difference between property rating and search-average rating	Base & CF
<code>review_diff_search</code>	Difference between review score and search-average review	Base & CF
<code>residual_price</code>	Property price minus its historical mean price	Base & CF
<code>location_score_gap</code>	Deviation of <code>prop_location_score1</code> from search-average	Base & CF
<code>location_score2_gap</code>	Deviation of <code>prop_location_score2</code> from search-average	Base & CF
<code>cheap_and_good</code>	Interaction: <code>price_rank</code> $\times$ <code>star_diff_search</code>	Base & CF
<code>search_size</code>	Number of properties in the search group	Base & CF
<code>price_per_night</code>	Total price divided by length of stay	Base & CF
<code>price_per_night_rel</code>	<code>price_per_night</code> relative to search-median	Base & CF
<code>prop_popularity</code>	Smoothed booking rate per property (computed on train, then mapped to test)	Base & CF
<code>dest_popularity</code>	Smoothed booking rate per destination (computed on train, then mapped to test)	Base & CF
<code>cf_score</code>	Destination-specific popularity score based on weighted booking and click history	CF only

Table 2: Summary of engineered features. Rows highlighted in yellow are learned on the training set and then copied into the test set. “Base&CF” indicates features used by both the LightGBM+XGBoost stack and the CatBoost CF pipeline; “CF only” denotes the collaborative-filtering score exclusive to the CF pipeline.

Collaborative Filtering (CF) Feature. CF is a recommendation technique that predicts user preferences by using historical interactions between other users and items [7]. To capture destination-specific user preferences, the feature `cf_score` was created. For each combination of `srch_destination_id` and `prop_id`, a weighted score was computed based on historical user behavior. Specifically, the `cf_score` represents the total number of weighted bookings and clicks for a given hotel in a given destination, thus reflecting how popular a property is among users traveling to that destination. To align with our target variable, we assigned that each hotel that was booked gets 5 points, and each click gets 1. The resulting `cf_score` illustrates the popularity of a property within a specific destination and indicates users’ interest. The `cf_score` was merged into the main dataset for *Strategy B* and used with other engineered features. In particular, it ranked among the most important features in our model’s feature importance analysis (Figure 6).

3.3 Feature Selection and Importance

For our feature engineering and selection, we made a feature evaluation pipeline consisting of the following models: *LightGBM*, *XGBoost* and *Lasso Logistic Regression*, all designed to predict if a hotel was booked or not (Binary classi-

fication). Each model ranked features by their importance, and based on that distribution, we decided which new features should stay in our dataset and which should not be added. The ones that ranked within the top 20 in at least two out of the three models were kept, while ones that ranked consistently low or contributed little additional information were excluded to reduce noise and over-fitting risk.

The process of how features were selected and evaluated is illustrated in Figure 5. The top 10 features are visualized in Figure 6. The `cf_score` dominates in all three evaluation models, followed by `random_bool`, `price_rank`, and `search_size`.

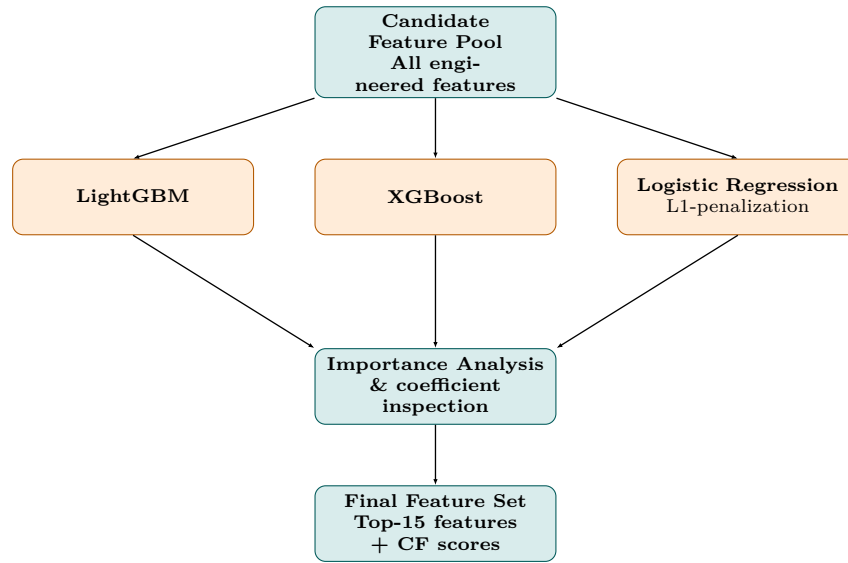


Fig. 5: Feature-selection pipeline: from the full engineered pool, evaluate in parallel with LightGBM, XGBoost, and L1-Logistic, combine via SHAP/coefficient importance analysis, and output the final chosen features.

4 Modeling

4.1 Pipeline

For the Expedia hotel ranking task we implemented two modeling pipelines, illustrated in Figure 7. The first is a stacked ensemble with LightGBM and XGBoost combined with RidgeCV on out-of-fold predictions. This ensemble was trained in three scenarios: (1) using only the base + engineered features from the dataset, (2) with additional dense feature representations from an unsupervised autoencoder, and (3) with representations from a supervised MLP trained to predict

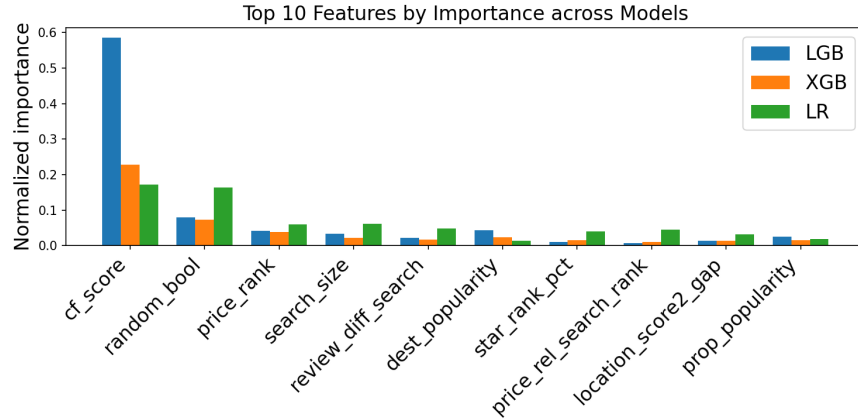


Fig. 6: Top 10 features by importance across LightGBM, XGBoost, and Logistic Regression. Feature importance is normalized per model.

booking outcomes. These embeddings were added to the original feature set to enrich the input. After any of these scenarios, a small Optuna hyperparameter search (10 trials on a 1-fold split) was used to tune both LightGBM (LambdaMART) and XGBoost (hist-based) models. These were then trained in a 5-fold group-wise setup with early stopping after 40 rounds. The out-of-fold predictions were blended via RidgeCV (with $\alpha=10$), and a final refit was performed on 100% of the data using the best-found parameters. In the second pipeline we used CatBoostRanker on the dataset with base + engineered features and also added the CF scores. We constructed a graded relevance target ($\text{clicks} + 5 \times \text{bookings}$), applied an 80/20 group split for early stopping, and then retrained on the full dataset using the best validation iteration. To evaluate the quality of our models, we relied on the normalized discounted cumulative gain at rank 5 (NDCG@5). This metric measures how well the top 5 predicted hotels align with the true hotel ranking for a given search. Both pipelines were designed to prevent overfitting. However, due to limited time and computational resources, we did not perform more extensive cross-validation and chose not to tune CatBoost, focusing instead on the more promising stacked ensemble pipeline.

4.2 Algorithms

In this section we will go more in depth into which algorithms we use, how they work and why we chose them.

First Model Explanation

LightGBM + XGBoost + RidgeCV Ensemble: The first model in this pipeline was a stacked ensemble combining LightGBM and XGBoost. we chose

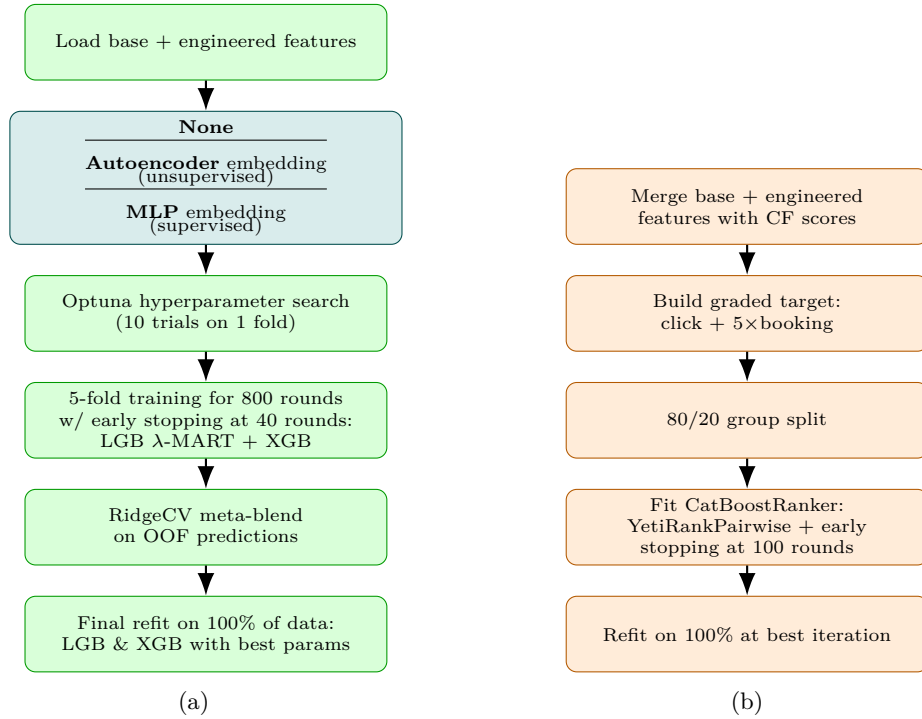


Fig. 7: Two modeling pipelines: (a) stacked LightGBM + XGBoost with embeddings, (b) CatBoost with CF features.

these models as both have shown strong performance on similar data tasks [4][3]. We thought combining them might allow their strengths to complement each other and improve overall model performance. Both models are based on gradient boosting, a technique that iteratively fits decision trees to correct the errors made by previous trees. LightGBM is particularly well-suited for large-scale ranking problems like this one because it uses histogram-based decision tree learning (which improves efficiency) and grows trees leaf-wise instead of level-wise, allowing it to focus on the most promising splits. In our implementation, LightGBM was configured with the lambdarank objective and evaluated using the ndcg@5 metric, which directly aligns with the ranking goal of our task. XGBoost complements LightGBM by offering stronger regularization through both L1 (Lasso) and L2 (Ridge) penalties, which help prevent overfitting and improve generalization. It was configured with the rank:ndcg objective and the histogram-based tree method (`tree_method="hist"`), also targeting ndcg@5 . To combine the strengths of both models, we used RidgeCV. This method learns the optimal linear combination of the LightGBM and XGBoost predictions by fitting a regularized linear model. It selects the best regularization strength using

cross-validation, making the final predictions more robust and stable. The fitted parameters of the ensemble are shown in Table 3.

Autoencoder (Unsupervised): To improve our base model, we added an unsupervised autoencoder to capture latent structures in our data as it is a common approach in these kind of tasks [8]. This neural network architecture compresses input features into a lower-dimensional representation and then tries to reconstruct the original input, thereby learning meaningful patterns in our data. For our model training, we extracted these embeddings generated by the autoencoder and added them the base + engineered features to enrich our dataset. The autoencoder consisted of two hidden layers with 256 and 128 neurons respectively, followed by a bottleneck layer with a size of 32 neurons (the latent space). The decoder had the same structure as the encoder, but mirrored. All layers used ReLU activations. We applied L2 regularization ($\lambda = 1e-4$) to all dense layers and added a dropout layer (dropout rate = 0.2) immediately after the bottleneck to reduce overfitting. The model was trained using the Adam optimizer and mean squared error (MSE) loss for up to 50 epochs, with early stopping (patience = 20) based on validation loss. We saved the best-performing model checkpoint and used it to generate the final embeddings added to the training and test data. The final epoch was 34 with a validation loss of 171502. The parameters of the fitted ensemble are shown in Table 3.

Multi Layer Perceptron (MLP) (Supervised to Predict Booking Outcomes): To try and increase our base model even further, we replaced the unsupervised autoencoder with a supervised MLP trained to predict booking outcomes. This approach is quite commonly used as well [9], and we hypothesized that it could increase task-specific embeddings when compared to an unsupervised AE. Instead of reconstructing inputs, the MLP directly learns features useful for distinguishing booked from non-booked items. The architecture consisted of an input layer followed by two dense layers with 128 and 64 ReLU units, then the embedding layer of 32 units followed and a final output layer with a sigmoid activation function predicted the booking probabilities. The model was trained using binary cross-entropy loss and the Adam optimizer for up to 50 epochs with a batch size of 4096. We applied early stopping on validation loss again (patience = 5) and saved the best checkpoint during training. After training, we added the embedding layer to our original features and continued training the original model. The MLP was trained for 18 epochs and achieved a validation loss of 0.1028. The ensemble parameters are depicted in Table 3. This model was also our final model for the Kaggle competition. We also tried running our final model with the CF-features to evaluate its effect.

Second Model Explanation

CatBoost with CF: For our second pipeline, we included the CF scores and trained a CatBoost model. We chose this model because it is particularly effective

Scenario	LightGBM Parameters	XGBoost Parameters	RidgeCV Weights (LGB / XGB)
Base	leaves: 110, lr: 0.0338, ff: 0.6177, minleaf: 73	xgb_lr: 0.0456, depth: 12	0.046 / 0.520
AE	leaves: 137, lr: 0.0119, ff: 0.6245, minleaf: 93	xgb_lr: 0.0421, depth: 8	0.089 / 0.022
MLP	leaves: 184, lr: 0.0218, ff: 0.7111, minleaf: 130	xgb_lr: 0.0941, depth: 6	0.094 / 0.030

Table 3: Best parameters and RidgeCV weights for the three LightGBM + XGBoost stacking scenarios. *lr* = learning rate, *ff* = feature fraction, *minleaf* = minimum data in leaf.

for datasets containing a mix of data types (such as our dataset), as it natively supports categorical variables without the need for manual encoding. It is also among the best performing algorithms on Kaggle for various similar competitions [5].

Just like LightGBM and XGBoost, CatBoost uses a gradient boosting approach to build its model. However, it differs in its technique to do this which is called ordered boosting. This technique helps reduce overfitting by preventing target leakage using permutations: instead of computing statistics (used for splitting decisions) on the full training set, it only uses preceding data points to compute these statistics. This ensures that no information from future examples leak into the training process and leads to less overfitting. We trained the model using the YetiRankPairwise loss function, which is designed to optimize pairwise ranking performance and aligns well with our task’s evaluation metric: NDCG@5. For this model we used the baseline parameters shown in table 4.

Model	lr	depth	l2	bag_temp	rand_str	iters	leaf_est_iters
CatBoost	0.05	8	3	1.0	1.0	1000	4

Table 4: Hyperparameters for the CatBoost model. Abbreviations: *lr* = learning rate, *l2* = L2 regularization, *bag_temp* = bagging temperature, *rand_str* = random strength, *iters* = iterations, *leaf_est_iters* = leaf estimation iterations.

5 Evaluation

Figure 8 compares validation and test NDCG@5 scores across all model pipelines. The ensemble of LightGBM, XGBoost, and RidgeCV that was used as a baseline, on NDCG@5 scored 0.5514 on a validation, and 0.3830 on a test set. Adding unsupervised autoencoder embeddings slightly improved performance on both validation and test, showing that general-purpose learned representations can improve the feature space. The best validation score (0.5918) was achieved using

a supervised MLP trained to predict booking outcomes. Similarly, this model also delivered the highest test score (0.3929), indicating strong generalization for the task at hand.

Notably, validation scores are consistently higher than test scores across all models, indicating overfitting. This difference likely comes from variations in the data distribution between the validation and test sets, possibly due to unseen user behavior or changes in features of properties. This overfitting suggests that the model likely has captured patterns specific to the validation set and is not generalizing well on the test set.

Including CF features led to a decrease in test performance, especially when combined with MLP embeddings (from 0.3929 to 0.3650). This suggests that CF features may have introduced noise, probably because they rely on historical user-item interactions that can be sparse and biased. The CatBoost + CF model similarly underperformed on the test set (0.3646), strengthening the idea that CF features were less effective in our setup.

The stacked LightGBM + XGBoost + RidgeCV ensemble with supervised MLP turned out to be our best performing model. This success could be explained by supervised MLP embeddings being trained directly to optimize the booking prediction task, ensuring that the learned representations are closely aligned with the target outcome. The model manages to find the features that are the most predictive of user behavior, which are relevant for ranking decisions. Unlike autoencoders, which preserve general structure, MLPs focus on extracting the most predictive patterns, resulting in stronger performance on NDCG@5.

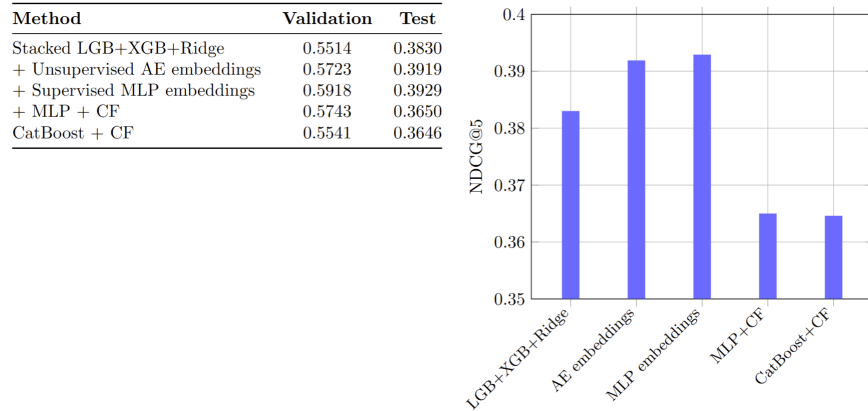


Fig. 8: Table and bar chart side by side, comparing Validation and Test NDCG@5 (bar chart shows Test only).

6 Deployment

To deploy our hotel ranking model at scale, Expedia would need to rely on a distributed infrastructure, since the data involved (clicks, bookings, user sessions, property metadata) is far too large to fit on a single machine. This can be handled using a distributed file system like HDFS, where data is chunked into large blocks and stored redundantly across many nodes [10]. This setup ensures fault tolerance and parallel access to large datasets for both training and inference.

Our pipeline could be distributed using a MapReduce-like approach. Each Map task could compute relevance scores for a specific search or destination, generating key-value pairs like (`search_id`, `hotel_score`). The Reduce step would then collect and sort these to select the top-ranked hotels per query. This style of processing is ideal for large-scale ranking and integrates well with tools like Apache Spark [11].

To retrain our ensemble model (LightGBM + XGBoost + RidgeCV), Expedia can parallelize both feature engineering and training across a compute cluster. For instance, each fold in cross-validation could be trained on a different machine. If more recent click and booking data becomes available, the same cluster setup allows for efficient nightly retraining, keeping the model up-to-date with changing user behavior.

Hash-based indexing techniques (e.g., hashing hotel IDs or search destinations) would help speed up feature retrieval. In practice, this means fewer input/output operations and faster response times during real-time ranking.

Our use of supervised embeddings (MLP) or autoencoders could also be further extended and retrained as new features or data become available.

6.1 Ethical AI Techniques

Bias Mitigation Strategy We identified selection bias in the training data due to the under-representation of independent hotels (`prop_brand_bool` = 0). The dataset exhibited a significant class imbalance, with branded hotels (part of larger chains) constituting 63.5% of instances compared to 36.5% for independent properties. Although the absolute difference in booking rates between chain and independent hotels was only 0.003 (2.9% vs. 2.6%), we still elected to under-sample branded properties to prevent any subtle bias in ranking scores. This skewed distribution risked training a model that disproportionately favored chain-affiliated hotels. To address this, we implemented under-sampling: randomly reducing branded hotel entries to match the count of independent ones. This approach preserved the predictive signal of the `prop_brand_bool` feature while mitigating imbalance. Removing the variable entirely would have retained the bias and obscured genuine relationships between hotel branding and outcomes.

We prioritized under-sampling over alternative methods like reweighting (which adjusts loss function penalties) for its simplicity and interpretability. While reweighting retains all data, it introduces complexity in tuning weight param-

eters and increases overfitting risks. Under-sampling provided direct balance without compromising model transparency, despite reducing dataset size.

Effectiveness Evaluation Post-mitigation analysis confirmed the strategy’s success. The balanced training data eliminated disparities in booking rates between hotel types, resolving the initial representation bias. Also the model performance remained competitive, with no significant degradation in accuracy. The use of stratified under-sampling avoided the negative effects of reweighting, such as overfitting, and preserved a link between hotel branding and predictions. This approach aligned with our fairness objectives while maintaining the model’s ability to capture meaningful patterns in the data.

Metric	Chain Hotels	Independent Hotels	Difference / Note
Before Balancing			
Booking Rate	0.029	0.026	0.003
Total Count	3,147,060	1,811,287	-
Booked Count	91,765	46,625	-
After Stratified Undersampling			
Booking Rate	0.026	0.026	0.000
Total Count	1,811,287	1,811,287	Balanced
Booked Count	46,625	46,625	Balanced

Table 5: Booking rates and counts for chain and independent hotels before and after balancing via stratified under-sampling.

7 What we learned

With this assignment, we learned how to build and evaluate ranking models using real world data [2]. We built on our knowledge of FE and learned to apply CF methods. If we had more time, we would have tried to think of more features and explore the literature in more detail. Additionally, we would run more Optuna trials for a more detailed hyperparameter search, and try to bring the gap between validation and test set closer for better generalization.

References

1. Wikipedia contributors. Discounted cumulative gain — wikipedia, the free encyclopedia, 2024. Accessed: 2025-05-25.
2. Vrije Universiteit Amsterdam. Dmt 2025: Data mining techniques assignment 2, 2025. Accessed: 2025-05-23.
3. Kaggle. Expedia hotel recommendations. <https://www.kaggle.com/c/expedia-hotel-recommendations/discussion/20225>, 2016. Accessed: 2025-05-23.
4. Kaggle. Otto – multi-objective recommender system. <https://www.kaggle.com/competitions/otto-recommender-system>, 2023. Accessed: 2025-05-23.
5. Kaggle. Which algorithms work best for kaggle competitions? <https://www.kaggle.com/questions-and-answers/483140>, 2025. Accessed: 2025-05-23.
6. Debraj Ghosh. Personalized ranking model for lodging. <https://medium.com/expedia-group-tech/personalized-ranking-model-for-lodging-5be43ae975fe>, 2021. Accessed: 2025-05-25.
7. J Ben Schafer, Dan Frankowski, Jon Herlocker, and Shilad Sen. Collaborative filtering recommender systems. In *The adaptive web: methods and strategies of web personalization*, pages 291–324. Springer, 2007.
8. Y. Li, X. Zhang, and L. Wang. Autoencoders and their applications in machine learning: a survey. *Artificial Intelligence Review*, 56:1–25, 2023.
9. Teklu Urgesa Etana Fikadu, Mrinal Das. Mlp-svm: A hybrid approach for improving the performance of the classification. *Springer Nature Applied Sciences*, 3(2):123–134, 2025.
10. Apache HDFS. Hadoop distributed file system, 2024. <https://hadoop.apache.org/docs/current/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>.
11. Apache Spark. Apache spark mllib, 2024. <https://spark.apache.org/mllib/>.